

Aula 29 - ATIVIDADES

MÃO NA MASSA: Construindo a Classe Modelo (Entidade)

Objetivo: Criar uma classe profissional para a entidade **Produto**, aplicando encapsulamento, tipagem estrita e validação de dados.

Passo 1: Criar o arquivo e definir Atributos Privados

Crie um arquivo chamado `Produto.php`. Utilizaremos `private` para garantir que os dados só possam ser alterados através de métodos seguros.

PHP

```
<?php
```

```
class Produto {  
    private string $nome;  
    private float $preco;  
    private int $estoque;  
}
```

Passo 2: Implementar o Construtor com Validação

O construtor garante que o objeto seja iniciado corretamente. Note que chamamos o método `setEstoque` dentro dele para aplicar a regra de validação já na criação.

PHP

```
public function __construct(string $nome, float $preco, int $estoque) {  
    $this->nome = $nome;  
    $this->preco = $preco;  
    $this->setEstoque($estoque);  
}
```

Passo 3: Criar Getters e Setters (Acesso Seguro)

Implementamos a lógica de proteção: o estoque nunca pode ser um número negativo. Se for tentado um valor inválido, o sistema emitirá um aviso.

PHP

```
// Recupera o valor do estoque  
public function getEstoque(): int {  
    return $this->estoque;  
}  
  
// Define o valor do estoque com validação  
public function setEstoque(int $quantidade): void {  
    if ($quantidade >= 0) {  
        $this->estoque = $quantidade;  
    } else {  
        echo "<b>Erro:</b> A quantidade ($quantidade) não pode ser negativa.<br>";  
    }  
}  
  
// Recupera o nome do produto  
public function getNome(): string {  
    return $this->nome;  
}
```

```

    }

    // Método extra: Formatação de preço para o padrão brasileiro
    public function exibirPrecoFormatado(): string {
        return "R$ " . number_format($this->preco, 2, ',', '.');
    }
} // Fim da classe

```

Passo 4: Criar o Script de Teste (index.php)

Agora, crie um arquivo chamado index.php para verificar se as suas regras de proteção estão a funcionar.

PHP

```

<?php
require_once "Produto.php";

// 1. Instanciando um produto com dados válidos
$p1 = new Produto("Monitor LED 24", 899.90, 15);

echo "<h2>Relatório do Produto</h2>";
echo "Nome: " . $p1->getNome() . "<br>";
echo "Preço: " . $p1->exibirPrecoFormatado() . "<br>";
echo "Estoque Atual: " . $p1->getEstoque() . "<br>";

echo "<hr>";

// 2. Testando a segurança (Tentativa de estoque negativo)
echo "Tentando alterar estoque para -5...<br>";
$p1->setEstoque(-5);

echo "<b>Estoque após tentativa inválida:</b> " . $p1->getEstoque();
// O resultado deve continuar 15, provando que a validação barrou o erro.
?>

```

Resultado Esperado: Ao abrir o `index.php` no navegador, deverá ver os dados do monitor, uma mensagem de erro destacada ao tentar definir o estoque como -5, e a confirmação de que o estoque permaneceu em 15.

EXERCÍCIOS DE FIXAÇÃO

Nível 1: Arrays e Coleções (Básico ao Intermediário)

- Arrays Indexados:** Crie um array chamado `$meses` contendo os nomes dos 12 meses do ano e exiba o mês correspondente ao índice 5.
- Manipulação Simples:** Adicione manualmente um novo item ao final de um array indexado de nomes de cidades.
- Array Associativo:** Crie um array `$perfil` com as chaves `nome`, `idade` e `cidade`. Exiba uma frase formatada usando esses dados.
- Alteração de Valores:** No array `$perfil` criado anteriormente, altere o valor da chave `idade` para um novo número.

5. **Cuidado com as Aspas:** Corrija o erro de sintaxe: `$config[versao] = 1.2;` (segundo a dica de boas práticas da aula).
6. **Case Sensitivity:** Explique o que acontece se você tentar acessar `$perfil['Nome']` sendo que a chave definida foi `'nome'`.
7. **Multidimensional Simples:** Crie um array `$estoque` que contenha dois outros arrays, cada um representando um produto com `nome` e `quantidade`.
8. **Acesso em Matriz:** No seu array `$estoque`, escreva o comando para exibir a quantidade do segundo produto.
9. **Estrutura de Tabela:** Monte um array multidimensional chamado `$usuarios` contendo 3 usuários, onde cada usuário é um array associativo (`login` e `perfil`).
10. **Inserção Dinâmica:** Adicione um 4º usuário ao array `$usuarios` utilizando a sintaxe de colchetes vazios `[]`.

Nível 2: Pilares da POO - Classes e Atributos

11. **Definição de Molde:** Escreva o código para uma classe vazia chamada `Veiculo`.
12. **Criação de Atributos:** Adicione os atributos públicos `marca`, `modelo` e `ano` à classe `Veiculo`.
13. **Instanciação:** Crie um objeto chamado `$meuCarro` que seja uma instância da classe `Veiculo`.
14. **Atribuição de Dados:** Defina valores para os atributos do objeto `$meuCarro` (ex: Toyota, Corolla, 2024).
15. **Independência de Objetos:** Crie um segundo objeto `$carroTrabalho` e atribua valores diferentes do primeiro. Explique por que alterar um não afeta o outro.
16. **Visibilidade:** Altere um dos atributos de `public` para `private` e tente acessá-lo fora da classe. O que acontece?

Nível 3: Pilares da POO - Métodos e Lógica

17. **Criação de Comportamento:** Na classe `Veiculo`, crie um método chamado `exibirDetalhes` que retorne uma string com a marca e o modelo do carro.
18. **Uso do \$this:** Dentro do método `exibirDetalhes`, utilize a pseudo-variável `$this` para acessar os atributos da classe.
19. **Método Construtor:** Adicione um método `__construct` à classe `Veiculo` para que ela receba a `marca` e o `modelo` no momento em que o objeto for criado com `new`.
20. **Lógica de Proteção (Encapsulamento):** Crie um método chamado `setAno` que receba um valor e só atualize o atributo `ano` se o número for maior que 1886 (ano da invenção do carro).

Desafio Integrador (Conexão com a SA)

Crie uma classe `Produto` (Atributos: `nome`, `preco`) e depois crie um **Array Multidimensional** onde cada índice do array armazene um **Objeto** da classe `Produto`.
